Chapitre 10 : Représentation des nombres en informatique

ITC - 2025/2026

Félix POTIÉ

Contexte 4
Représentation des nombres en base 5
1 - Les entiers naturels en binaire 7
1.1 - Binaire vers entier base 10 8
1.2 - Entier base 10 vers binaire 10
1.3 - Hexadécimal 12
2. Négatifs 13
2.1 - Négatifs : bit de signe 14
2.2 - Négatifs : Complément à 2 15
3. Réels 23
3.1 - Réels : Problème 24
3.2 - Réels : norme IEEE-754 25
3.3 Réels : Arrondis 35

3.4 Decimal vers IEEE-754 37
4. En python 39
5. Round() 40

Contexte

Objectif : découvrir comment sont représentés les nombres en mémoire.

Comprendre comment sont représentés les nombres en informatique vous permettra peut-être un jour d'éviter de porter la responsabilité de l'explosion d'une fusée comme Ariane 5, dont l'explosion fut d'origine numérique.

Représentation des nombres en base

Dans notre quotidien, nous utilisons généralement des nombres en **base 10** (grâce aux chiffres de 0 à 9).

Pour fonctionner un ordinateur reçoit des signaux électriques qui sont soit 1 lorsqu'il y a du courant, 0 sinon. Toute information, qu'il s'agisse d'un nombre, d'un texte ou d'une image, est finalement codée sous forme de bits.

Dans ce cours on va se focaliser sur la représentation des nombres.

On va donc avoir besoin de transformer nos entiers base 10 en **base 2**, aussi appelé système **binaire**.

Un chiffre en binaire est appelé un **bit**.

Représentation des nombres en base - 2

Notation 1: Base d'un nombre.

Pour définir la base d'un nombre, on note :

$N_{(\mathrm{b})}$ où N est le nombre et b la base correspondante.

ex : $101_{(2)} = 5_{(10)} \neq 101_{(10)}$

1 - Les entiers naturels en binaire

Comment représenter les entiers naturels en binaire ?

Nombre en décimal	Nombre en binaire
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111

1.1 - Binaire vers entier base 10

Définition 1: Représentation binaire.

Une séquence $b_{n-1}b_{n-2}...b_1b_0$ de n bits b_i correspond au nombre N en base 10 suivant :

$$N = \sum_{0 \leq i < n} b_i \times 2^i$$

Question : Transformer en base 10 les entiers naturels en binaire suivants :

- $1010_{(2)}$
- $1100\ 0100_{(2)}$
- $1\ 0000\ 0000_{(2)}$
- $1111\ 1111_{(2)}$

1.1 - Binaire vers entier base 10 - 2

Réponse :

- $1010_{(2)} = 2^3 + 2^1 = 8 + 2 = 10_{(10)}$
- $1100\ 0100_{(2)} = 2^7 + 2^6 + 2^2 = 128 + 64 + 4 = 196_{(10)}$
- $1\ 0000\ 0000_{(2)} = 2^8 = 256_{(10)}$
- $1111\ 1111_{(2)} = \sum_{i=0}^{7} 2^i = 2^8 - 1 = 256 - 1 = 255_{(10)}$

Définition 2: Octet.

Un octet est un ensemble de 8 bits.

Question : Sur un octet, quel est le nombre maximum que l'on peut écrire ?

1.2 - Entier base 10 vers binaire

Algorithme 1: entier base 10 vers binaire.

Soit $N \in \mathbb{N}^*$, on cherche son équivalent en base 2 :

- On fait la division euclidienne de N par 2
- N devient le quotient de cette division
- On recommence tant que le quotient ne vaut pas 0.
- Le nombre en binaire apparaît alors : il suffit de prendre tous les restes de bas en haut.

Question : Trouver la représentation de $185_{(10)}$ en binaire.

1.2 - Entier base 10 vers binaire - 2

185 = 2 x 92 + 1

92 = 2 x 46 + 0

46 = 2 x 23 + 0

23 = 2 x 11 + 1

11 = 2 x 5 + 1

5 = 2 x 2 + 1

2 = 2 x 1 + 0

1 = 2 x 0 + 1

Donc $185_{(10)} = 1011\ 1001_{(2)}$

1.3 - Hexadécimal

En informatique il y a une autre base qui est souvent utilisée. La base 16 autrement appelée l'hexadécimal. On retrouve tous les chiffres de 0 à 9 auxquels on ajoute les lettres :

$A_{(16)} = 10_{(10)}, B_{(16)} = 11_{(10)}, C_{(16)} = 12_{(10)},$

$D_{(16)} = 13_{(10)}, E_{(16)} = 14_{(10)}, F_{(16)} = 15_{(10)}$

Cette base est surtout utile pour lire plus facilement les paquets de bits. Un octet peut donc s'écrire en 2 caractères en héxadécimal. Ce qui est plus lisible et permet d'éviter des erreurs humaines.

Par exemple on a des 2 octets suivants :

$$\begin{array}{cccc} 0101 & 1011 & 1111 & 1001_{(2)} \\ 5 & B & F & 9_{(16)} \end{array}$$

2. Négatifs

Jusqu'ici, nous avons vu comment représenter des entiers naturels en binaire. Cependant, dans la plupart des applications informatiques, il est nécessaire de représenter également des entiers négatifs.

Des idées sur comment on pourrait faire ?

2.1 - Négatifs : bit de signe

Une première idée consiste à réserver un bit pour le signe : 0 : nombre positif 1 : nombre négatif

Par exemple, sur 8 bits :

0000 0101 $\longrightarrow +5_{(10)}$

1000 0101 $\longrightarrow -5_{(10)}$

Problèmes de cette méthode :

- existence de deux représentations de 0 (+0 et −0),
- complexité des opérations arithmétiques

Exemple : 1000 0101 + 0000 0101

2.2 - Négatifs : Complément à 2

Le **complément à 2** est la méthode utilisée par les ordinateurs pour représenter les entiers signés.

Définition 3: Complément à 2.

En complément à 2, on représente sur n bits les entiers, les valeurs possibles sont donc dans l'intervalle $[-2^{n-1}, 2^{n-1} - 1]$

Un mot binaire $b_{n-1}b_{n-2}...b_0$ représente la valeur :

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

2.2 - Négatifs : Complément à 2 - 2

Algorithme 2: Complément à 2.

- Pour x un entier **positif ou nul**:
  - écrire la représentation binaire de x sur n bits.
- Pour $-x$ un entier **négatif** :
  - écrire la représentation binaire de x sur n bits,
  - inverser tous les bits (complément à 1),
  - ajouter 1 au résultat.

Question :

Donner la représentation en complément à 2 de 5 et -5 sur 8 bits.
Puis additionner ces représentations, que remarquez-vous?

2.2 - Négatifs : Complément à 2 - 3

Réponse :

- On commence par $5_{(10)} = 0000\ 0101_{(2)}$
- Complément à 1 : $0000\ 0101 \rightarrow 1111\ 1010$
- Ajout de 1 : $1111\ 1011_{(2)} = -5_{(10)}$

Addition :

$$\begin{array}{rr} & 0000\ 0101 \\ & +1111\ 1011 \\ \hline 1 & 0000\ 0000 \end{array}$$

On a dépassé des 8 bits, on ignore le dépassement, on a donc bien 0 en faisant 5-5.

2.2 - Négatifs : Complément à 2 - 4

Algorithme 3: Complément à 2 vers base 10.

- Si le bit de poids fort est 0 :
  - écrire la représentation base 10 de la représentation binaire.
- Sinon :
  - inverser tous les bits (complément à 1),
  - ajouter 1 au résultat.
  - écrire la représentation base 10 du binaire obtenue
  - Ajouter le signe “-”

Question :

On vous donne la représentation en complément à 2 sur 8 bits suivante : 1001 0101. Trouver sa valeur en base 10.

2.2 - Négatifs : Complément à 2 - 5

1. Bit de poids fort égal à 1, on retient qu'on a un nombre négatif. On fait le complément à 1 : 1001 0101 $\rightarrow$ 0110 1010
2. On ajoute 1 : 0110 1010 $\rightarrow$ 0110 1011
3. On convertit en base 10 :
   $2^0 + 2^1 + 2^3 + 2^5 + 2^6 = 1 + 2 + 8 + 32 + 64 = 107$
4. Résultat final : 1001 0101 en complément à 2 représente -107 en base 10.

2.2 - Négatifs : Complément à 2 - 6

Définition 4: Overflow / Dépassements de capacité.

On parle de dépassement de capacité lorsqu'une opération arithmétique produit un résultat hors de l'intervalle représentable sur n bits.

Question :

Donner la représentation sur 8 bits de $100_{(10)}$ en complément à 2.
Puis additionner $100_{(10)} + 100_{(10)}$ en complément à 2 sur 8 bits.
Que donne le résultat ?

2.2 - Négatifs : Complément à 2 - 7

$100_{(10)} = 0110\ 0100_{(2)}$

$$\begin{array}{r} 0110\ 0100 \\ +0110\ 0100 \\ \hline 1100\ 1000 \end{array}$$

Ce qui nous donne un résultat négatif, cela car 200, n'est pas représentable sur 8 bits.
En effet 200 n'est pas dans l'intervalle $[-128, 127]$.

2.2 - Négatifs : Complément à 2 - 8

Remarque importante :

L'overflow n'est un problème que lorsque **les deux opérandes ont le même signe**.

Lorsqu'on additionne un nombre positif et un nombre négatif, le dépassement de capacité est **voulu** et **attendu**. C'est le mécanisme qui permet au complément à 2 de fonctionner correctement : on ignore simplement le bit de débordement, ce qui donne le bon résultat. (ex : $5_{(10)} + (-5)_{(10)}$ page 16)

3. Réels

Les représentations vues jusqu'ici permettent de coder des entiers sur un nombre fini de bits.
Cependant, de nombreuses applications nécessitent la manipulation de nombres réels.

3.1 - Réels : Problème

```
>>> print(0.3 == 0.1 + 0.1 + 0.1)
False
>>>
>>> print(0.1+0.1+0.1)
0.3000000000000004
```

Il est impossible de représenter exactement tous les nombres réels avec un nombre fini de bits. On utilise donc une représentation approchée, standardisée par la norme IEEE 754.

3.2 - Réels : norme IEEE-754

Définition 5.

Un nombre flottant est représenté sous la forme :

$$(-1)^s \times m \times 2^{e-d}$$

où :

- s est le bit de signe,
- m est la mantisse (ou significande),
- e est l'exposant, d est ce qu'on appelle le biais, il permet d'avoir un exposant négatif sans passer par le complément à 2.

3.2 - Réels : norme IEEE-754 - 2

La norme IEEE 754 définit plusieurs formats. Les plus courants sont :

Format	Taille	Signe	Exposant	Mantisse
simple précision	32 bits	1 bit	8 bits (biais 127)	23 bits
double précision	64 bits	1 bit	11 bits (biais 1023)	52 bits

3.2 - Réels : norme IEEE-754 - 3

Dans la suite, on se place en simple précision (32 bits).

1	8	23
signe	exposant	mantisse

- Signe s : 0 positif, ou 1 négatif
- Mantisse m est toujours comprise dans l'intervalle [1, 2[, Ainsi, si les 23 bits dédiés à la mantisse sont $b_1 b_2 ... b_{23}$, elle est de la forme:

$$m = 1 + b_1 \cdot 2^{-1} + b_2 \cdot 2^{-2} + \cdots + b_{23} \cdot 2^{-23}$$

.

- Exposant : le biais est de 127

3.2 - Réels : norme IEEE-754 - 4

$$\overbrace{1}^{signe}\ \overbrace{10000110}^{exposant}\ \overbrace{10101101100000000000000}^{fraction}$$

$$\begin{aligned}
\text{signe} &= (-1)^1 \\
&= -1
\end{aligned}$$

$$\begin{aligned}
\text{exposant} &= (2^7 + 2^2 + 2^1) - 127 \\
&= (128 + 4 + 2) - 127 \\
&= 134 - 127 \\
&= 7
\end{aligned}$$

$$\begin{aligned}
\text{mantisse} &= 1 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-6} + 2^{-8} + 2^{-9} \\
&= 1{,}677734375
\end{aligned}$$

$$-1{,}677734375 \times 2^7 = -214{,}75$$

3.2 - Réels : norme IEEE-754 - 5

Question : Déterminer la valeur flottante représentée par le mot binaire suivant en simple précision :
$1100\ 0001\ 0010\ 0100\ 0000\ 0000\ 0000\ 0000_{(2)}$

Rappel :

$(-1)^s \times m \times 2^{e-127}$

Signe : 1 bit (positif ou negatif)

Exposant : 8 bits (valeur decimal -biais de 127)

Mantisse : 23 bits $\left(m = 1 + \sum_{i=1}^{23} b_i \cdot 2^{-i}\right)$

3.2 - Réels : norme IEEE-754 - 6

Réponse :

- Signe $s = 1$ donc le nombre est négatif.
- Exposant $e = 1000\ 0010_{(2)} = 130_{(10)}$ donc $e - d = 130 - 127 = 3$
- Mantisse : 010 0100 0000 0000 0000 0000

$m = 1 + 1 \cdot 2^{-2} + 1 \cdot 2^{-5} = 1 + 0.25 + 0.03125 = 1.28125$

$1100\ 0001\ 0010\ 0100\ 0000\ 0000\ 0000\ 0000_{(2)}$

représente donc le nombre :

$(-1)^1 \times 1.28125 \times 2^3 = -10.25$ en simple précision.

3.2 - Réels : norme IEEE-754 - 7

Question : Déterminer le plus grand flottant positif représentable en simple précision.

3.2 - Réels : norme IEEE-754 - 8

Réponse :

- Plus grand nombre flottant positif :
  - Exposant maximal : $e = 254$ donc $e - d = 127$
  - Mantisse maximale : $m = 1 + \sum_{i=1}^{23} 2^{-i} = 1 + 1 - 2^{-23}$

  Donc le plus grand nombre flottant positif est :
  $(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$ en simple précision.
  En double précision on trouve environ 1.8×10^{308}

  On peut le vérifier en console python (64 bits):

```
>>> 1.7*10**308
1.7e+308
>>> 1.8*10**308
inf
```

3.2 - Réels : norme IEEE-754 - 9

Certaines valeurs sont utilisées pour des cas spéciaux :

signe	exposant	fraction	valeur spéciale
0	0	0	$+0$
1	0	0	-0
0	255	0	$+\infty$
1	255	0	$-\infty$
0	255	$\neq 0$	NaN

3.2 - Réels : norme IEEE-754 - 10

- Plus petit nombre flottant positif est donc :
  - Exposant minimal : $e = 1$ donc $e - d = -126$
  - Mantisse minimale : $m = 1$

  Donc le plus petit nombre flottant positif normalisé est :
  $1 \times 2^{-126} \approx 1.18 \times 10^{-38}$ en simple précision.
  En double précision, on trouve environ 2.2×10^{-308}.

3.3 Réels : Arrondis

Il est important de noter que la représentation des nombres décimaux par des flottants est une représentation approximative. Par exemple, le nombre décimal $1,6$ ne peut pas être représenté exactement avec cet encodage.

Par exemple le nombre flottant le plus proche de 1,6 est :

$$0\ 01111111\ 10011001100110011001101$$

qui correspond au nombre décimal

$$\begin{array}{rr} & (2^{-1}+2^{-4}+2^{-5}+2^{-8}+2^{-9}+2^{-12}+2^{-13}+ \\ & 2^{-16}+2^{-17}+2^{-20}+2^{-21}+2^{-23}) \\ \times & 2^{(127-127)} \\ \hline = & 1{,}60000002384185791015625 \end{array}$$

3.3 Réels : Arrondis - 2

Autre exemple, avec un flottant sur 32 bits, on ne peut pas représenter exactement le nombre décimal 299795458 (vitesse de la lumière) :

```
>>> import numpy as np
>>> np.float32(299795458)
299795460.0
>>>
```

3.4 Decimal vers IEEE-754

Question : Coder le nombre 125,75 avec la norme IEEE-754 sur 32 bits.

Pas besoin de connaitre par coeur :

Algorithme 4: Coder un nombre réel en simple précision.

1. On trouve le bit de signe
2. On convertit la partie entière en binaire
3. On convertit la partie décimale en binaire
4. On normalise le nombre (on le met sous la forme $1,\ldots. \times 2^e$)
5. On calcule l'exposant (en prenant en compte le biais)
6. On détermine la mantisse

3.4 Decimal vers IEEE-754 - 2

1. Signe $s = 0$ donc le nombre est positif.
2. Conversion en binaire de la partie entière :
   $125_{(10)} = 111\ 1101_{(2)}$
3. Conversion en binaire de la partie décimale :
   $0,75_{(10)} = 0,11_{(2)}$
4. Normalisation : on écrit sous la forme $1,.... \times 2^e$:
   $125,75_{(10)} = 111\ 1101,11_{(2)} = 1,11110111 \times 2^6$
5. Exposant $e = 6 + 127 = 133_{(10)} = 1000\ 0101_{(2)}$
6. Mantisse $m = 11110111000000000000000_{(2)}$ (sur 23 bits)

Donc la représentation en simple précision de $125,75_{(10)}$ est :
$0\ 10000101\ 11110111000000000000000_{(2)}$

4. En python

Les entiers (int) :

- 64 bits en double précision : $[-2^{63}, 2^{63} - 1]$
- Pour les entiers plus grands : stockage sous forme de liste de chiffres en base 10, ce qui permet de représenter des entiers de taille arbitraire (tant que la mémoire le permet).

Les flottants (float) :

- 64 bits en double précision avec la norme IEEE 754 : $[-2^{1023}, 2^{1023} - 1]$

5. Round()

Définition 6: Fonction round() en Python.

La fonction `round()` arrondit un nombre flottant à un nombre spécifié de décimales.

Syntaxe : `round(nombre, ndigits)`

- `nombre` : le flottant à arrondir
- `ndigits` : nombre de décimales (optionnel, par défaut 0)

Pourquoi l'utiliser ?

- Corriger les erreurs d'arrondi dues à la représentation IEEE 754
- Afficher des résultats lisibles avec un nombre de décimales approprié
- Éviter les comparaisons inexactes entre flottants (ex : `0.1 + 0.2 == 0.3` est `False` en Python)